

**Desempenho de Sistemas de Filas Simulados: Proposta de Experimento
Fatorial 2^3 com Replicações e Análise Exploratória dos Dados**

Gruner Antonio Sanchez Morales

Departamento de Computação

Universidade Federal de Ouro Preto

Projeto e Análise de Experimentos Computacionais — Entrega 1

21 de setembro de 2026

Resumo

Este relatório apresenta a proposta de um experimento computacional e a análise exploratória dos dados que o fundamenta. O problema investigado é o efeito de três fatores — número de servidores, taxa de chegada e distribuição do tempo de serviço — sobre a espera média em fila de um sistema simulado por eventos discretos com chegadas de Poisson. O estudo usará um projeto fatorial 2^3 com $r = 10$ replicações independentes por combinação (80 corridas), com dados gerados em SimPy e com sementes e ordem de execução aleatorizadas. A análise exploratória, feita com corridas piloto de uma única configuração e separada do experimento, indicou que o simulador reproduz os valores teóricos (erro relativo máximo de 1,3% na verificação), que um aquecimento de 1.000 clientes é adequado, que as réplicas são independentes e que a média por corrida tem distribuição assimétrica à direita, o que exigirá atenção aos resíduos e possivelmente uma transformação logarítmica. As 80 corridas exigem cerca de 36 s de processamento, portanto o experimento é viável. A hipótese geral, formulada antes das corridas, será avaliada somente com o fatorial completo.

Palavras-chave: experimento fatorial, simulação de eventos discretos, teoria de filas, análise exploratória de dados, reprodutibilidade

Desempenho de Sistemas de Filas Simulados: Proposta de Experimento

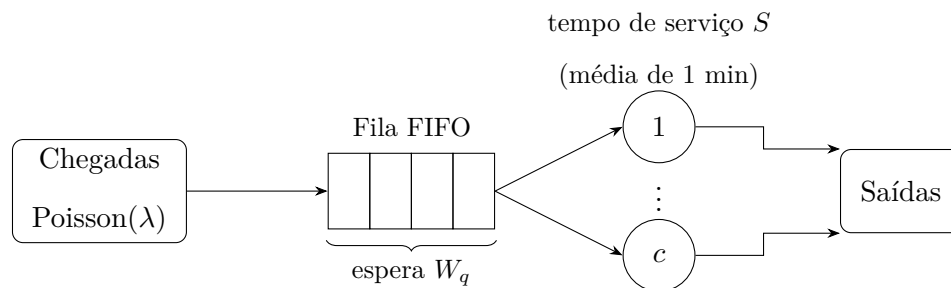
Fatorial 2^3 com Replicações e Análise Exploratória dos Dados

Contextualização e Definição do Problema

Filas aparecem sempre que uma demanda variável disputa recursos limitados: requisições em servidores web, pacotes em roteadores, processos em uma CPU ou clientes em um guichê. Em sistemas computacionais, o tempo de espera é um componente central da latência percebida e o projetista precisa decidir quanta capacidade alocar para uma carga esperada (Jain, 1991). Essa decisão envolve pelo menos três aspectos: quantos servidores usar, quão intensa é a chegada de trabalho e quão variável é o tempo de atendimento.

A teoria de filas oferece fórmulas fechadas para casos particulares, como as filas $M/M/1$, $M/M/c$ e $M/G/1$, mas o método deixa de ser prático quando o modelo se afasta desses casos. Nessas situações, a simulação de eventos discretos permite estudar o sistema por meio de experimentos computacionais (Law & Kelton, 2000). Como cada corrida de simulação é um resultado aleatório, o estudo precisa de planejamento experimental: escolha de fatores e níveis, replicações independentes, aleatorização e análise estatística dos resultados (Jain, 1991; Montgomery, 2017). Um projeto fatorial completo é preferível a variar um fator por vez porque estima, com o mesmo conjunto de corridas, os efeitos principais e as interações entre os fatores.

A Figura 1 mostra o sistema estudado: clientes chegam segundo um processo de Poisson, esperam em uma fila única com disciplina FIFO e são atendidos por c servidores idênticos.

Figura 1*Sistema de filas modelado*

Nota. Clientes chegam com taxa λ (clientes por minuto). O tempo que passam na fila antes de iniciar o serviço é a espera W_q , a variável de resposta do experimento.

Problema

Como o número de servidores, a intensidade de chegadas e a variabilidade do tempo de serviço afetam, isoladamente e em conjunto, a espera média em fila de um sistema em regime estacionário?

Relevância

A pergunta tem relevância prática, porque orienta o dimensionamento de capacidade, e relevância teórica, porque a espera em filas responde de forma não linear à utilização do servidor: perto da saturação, pequenas mudanças de carga têm grande efeito, o que sugere interações entre os fatores. Do ponto de vista metodológico, o estudo reúne os cuidados de um experimento computacional bem conduzido: verificação do simulador, tratamento do período de aquecimento, replicações independentes e reprodutibilidade (Sandve et al., 2013; Wilson et al., 2017).

Objetivo

O objetivo geral é avaliar, por meio de um projeto fatorial 2^3 com $r = 10$ replicações, o efeito do número de servidores, da taxa de chegada e da distribuição do tempo de serviço sobre a espera média em fila em um sistema simulado. Os objetivos específicos são:

- (a) estimar os efeitos principais e as interações dos três fatores, com intervalos de confiança;

- (b) determinar a proporção da variação total atribuída a cada fator, a cada interação e ao erro experimental;
- (c) verificar as suposições do modelo estatístico (normalidade, homocedasticidade e independência dos resíduos) e decidir sobre uma transformação da resposta;
- (d) garantir que todo o experimento possa ser reproduzido a partir de código, sementes e versões de software registrados.

Hipótese Geral do Estudo

A hipótese foi definida antes da execução do experimento fatorial e será avaliada por meio dele. Ela é uma expectativa teórica: não foi construída a partir dos dados do piloto (seção “Análise Exploratória dos Dados”) nem é avaliada nele. Ela se apoia na teoria de filas: para uma fila M/G/1, a espera média é $W_q = \rho(1 + c_s^2)/[2\mu(1 - \rho)]$, em que ρ é a utilização do servidor, μ a taxa de serviço e c_s o coeficiente de variação do tempo de serviço (Jain, 1991).

Hipótese geral. A espera média em fila diminui quando aumenta o número de servidores (A), aumenta quando cresce a taxa de chegada (B) e diminui quando o tempo de serviço deixa de ter variabilidade (C, nível constante). Os efeitos de A e de C serão mais intensos sob alta carga, isto é, existirão interações A×B e B×C.

Formalização. Nesta entrega a hipótese é enunciada em palavras. Na análise do fatorial, cada efeito q_j , com $j \in \{A, B, C, AB, AC, BC, ABC\}$, será avaliado com intervalo de confiança e teste de significância ($\alpha = 0,05$). Como W_q é o produto de um termo que depende de ρ e de um termo que depende de c_s^2 , espera-se, por teoria, que o modelo aditivo se ajuste melhor à resposta em escala logarítmica.

Estrutura Experimental

Unidade experimental

A unidade experimental é uma corrida independente do simulador: $N = 50.000$ clientes gerados com uma semente própria, a partir do sistema vazio.

Fatores e níveis

A Tabela 1 resume os três fatores, cada um com dois níveis.

Tabela 1*Fatores e níveis do experimento*

Fator	Descrição	Nível -1	Nível $+1$
A	Número de servidores (c)	1	2
B	Taxa de chegada (λ , clientes/min)	0,5	0,9
C	Distribuição do tempo de serviço	Exponencial	Constante

Nota. Em ambos os níveis de C o tempo médio de serviço é 1 min; a distribuição constante tem variabilidade nula ($c_s = 0$) e a exponencial tem $c_s = 1$.

Variável de resposta

A resposta é a espera média em fila por corrida, em minutos, calculada sobre os clientes que chegam depois do aquecimento:

$$\overline{W}_q = \frac{1}{N - W} \sum_{i=W+1}^N W_{q,i}, \quad (1)$$

com $N = 50.000$ e $W = 1.000$ clientes descartados. A resposta é medida em tempo simulado; portanto, não depende da velocidade do computador.

Condições mantidas constantes

Permanecem fixos: o tempo médio de serviço (1 min); a disciplina FIFO; a fila de capacidade infinita e sem abandonos; o processo de chegada de Poisson; o estado inicial (sistema vazio); os valores de N e W ; o código do simulador e as versões de software; e a máquina de execução, sem outras cargas pesadas.

Aleatorização

O procedimento tem três partes. Primeiro, cada uma das 80 corridas recebe uma sequência de números pseudoaleatórios independente, obtida com `SeedSequence.spawn` do NumPy a partir de uma semente-mestre registrada; o gerador é o PCG64 (O'Neill, 2014). Segundo, a ordem de execução das 80 corridas é definida por uma permutação uniforme, com semente própria, para que efeitos de ordem (como aquecimento do processador ou variação da carga da máquina) não se confundam com os fatores.

Terceiro, o plano completo é gerado e guardado antes de qualquer corrida do experimento (Apêndice A).

Projeto Fatorial 2^3r

O projeto contém todas as $2^3 = 8$ combinações dos níveis, cada uma com $r = 10$ replicações independentes, o que dá 80 corridas. A Tabela 2 apresenta as combinações na ordem padrão, com a utilização do servidor $\rho = \lambda/(c\mu)$; em todas elas $\rho < 1$, de modo que existe regime estacionário.

Tabela 2

Combinações experimentais do projeto 2^3 ($r = 10$ replicações por combinação)

Célula	A	B	C	c	λ (cl/min)	Serviço	ρ
1	−	−	−	1	0,5	exponencial	0,50
2	+	−	−	2	0,5	exponencial	0,25
3	−	+	−	1	0,9	exponencial	0,90
4	+	+	−	2	0,9	exponencial	0,45
5	−	−	+	1	0,5	constante	0,50
6	+	−	+	2	0,5	constante	0,25
7	−	+	+	1	0,9	constante	0,90
8	+	+	+	2	0,9	constante	0,45

Nota. − e + indicam os níveis codificados de cada fator (Tabela 1). ρ é a utilização de cada servidor. Total: $8 \times 10 = 80$ corridas.

O modelo estatístico é o modelo de regressão com variáveis codificadas

$x_A, x_B, x_C \in \{-1, +1\}$ (Jain, 1991):

$$y_{ij} = q_0 + q_A x_A + q_B x_B + q_C x_C + q_{AB} x_A x_B + q_{AC} x_A x_C + q_{BC} x_B x_C + q_{ABC} x_A x_B x_C + e_{ij}, \quad (2)$$

em que y_{ij} é a resposta da replicação j da combinação i e e_{ij} é o erro experimental. A análise, que será feita somente depois da execução das 80 corridas, terá os seguintes passos: estimação dos efeitos pela tabela de sinais; alocação da variação; análise de variância; intervalos de confiança de 95% para os efeitos; e diagnóstico dos resíduos (resíduos contra valores ajustados e gráfico Q-Q). Se os resíduos indicarem variância

não constante, será aplicada uma transformação da resposta, como o logaritmo, escolhida entre as da família de Box e Cox (1964) (ver também Montgomery, 2017).

Réplicas independentes. Cada réplica é uma execução completa da combinação: o sistema parte vazio, recebe uma semente nova e simula N clientes. Não são réplicas partes de uma mesma corrida (por exemplo, lotes de uma corrida longa) nem clientes consecutivos, cujas esperas são dependentes (seção “Visualizações”). A independência entre réplicas foi verificada no piloto (Figura 4).

Fonte dos Dados

Os dados serão gerados por simulação; não há base pública nem coleta direta. O simulador implementa o sistema da Figura 1 como um modelo de eventos discretos em SimPy (Lünsdorf & Scherfke, s.d.): cada cliente é um processo que solicita um servidor (**Resource**, com fila FIFO), registra a espera e ocupa o servidor durante o tempo de serviço. Os tempos entre chegadas são exponenciais com taxa λ , e os tempos de serviço são exponenciais ou constantes, com média de 1 min. Os números aleatórios de cada corrida são gerados antes da simulação, de modo que a corrida fica determinada pela sua semente.

Verificação do simulador

Antes do piloto, o simulador foi verificado contra fórmulas teóricas: Pollaczek–Khinchine (M/M/1 e M/D/1) e Erlang C (M/M/2). Foram usadas 5 corridas longas de 400.000 clientes para cada uma das seis combinações com fórmula fechada; o maior erro relativo da média foi 1,3%. As combinações M/D/2 não têm fórmula fechada simples; o código que as executa é o mesmo das combinações verificadas, e só muda o vetor de tempos de serviço.

Ambiente Computacional

As corridas serão executadas em Python com as bibliotecas da Tabela 3: SimPy para a simulação (Lünsdorf & Scherfke, s.d.), NumPy para a geração de números aleatórios (Harris et al., 2020), SciPy e statsmodels para a estatística (Seabold & Perktold, 2010; Virtanen et al., 2020), pandas para os dados (McKinney, 2010) e Matplotlib para os gráficos (Hunter, 2007). Os documentos são compilados com pdfTeX

3.141592653-2.6-1.40.25 (TeX Live 2023/Debian) e Biber 2.19. O piloto foi executado em uma máquina com processador Intel(R) Xeon(R) Processor @ 2.10GHz (1 núcleo(s) lógico(s)), 3,9 GB de memória e sistema Linux 6.18.44-fc-v37.

Tabela 3

Versões de software usadas no piloto

Componente	Versão
Python	3.12.3
SimPy	4.1.2
NumPy	2.4.4
SciPy	1.17.1
pandas	3.0.2
statsmodels	0.15.0
Matplotlib	3.10.8

Nota. As versões exatas estão registradas em `codigo/requirements.txt`.

Para que o ambiente seja controlado e o experimento reproduzível (Sandve et al., 2013; Wilson et al., 2017), foram adotadas as seguintes medidas: versões de software fixadas; sementes registradas em `config.py`; execução exclusivamente por *scripts*, sem passos manuais; dados brutos das 80 corridas preservados em arquivo; e uma única máquina, sem outras cargas pesadas, para todas as corridas. A estrutura de arquivos e os comandos de reprodução estão no Apêndice B.

Compatibilidade com os recursos. O total de $2^3 \times 10 = 80$ execuções é compatível com o hardware: o piloto levou cerca de 0,45 s por corrida, ou 36 s para as 80 corridas. O experimento não usa APIs nem modelos de linguagem, portanto não há cotas a considerar.

Análise Exploratória dos Dados

A análise exploratória (AED) usa corridas piloto de uma única configuração: a célula 3 do projeto (um servidor, $\lambda = 0,9$, serviço exponencial; $\rho = 0,90$). Ela foi escolhida antes de qualquer corrida como a mais exigente para dimensionar o

experimento, pois tem a maior utilização, em que o transiente inicial é mais longo e a variância da média é maior (Kelton & Law, 1985; Law & Kelton, 2000). Foram simuladas 30 corridas independentes, cada uma uma execução completa (sistema vazio, 50.000 clientes, semente própria), com $W = 1.000$. As sementes são diferentes das do experimento, e o piloto não entra na análise fatorial. Esta seção caracteriza a variável de resposta e avalia a viabilidade do experimento; não compara níveis dos fatores, não estima efeitos e não avalia a hipótese geral (seção “Hipótese Geral do Estudo”), que só será avaliada com o fatorial completo.

Estatísticas descritivas

A Tabela 4 resume as 30 médias por corrida. A média do piloto (9,112 min) ficou próxima do valor teórico da fila M/M/1 (9,00 min): o erro relativo foi de 1,2% e o intervalo de confiança de 95% (8,68 a 9,55 min) contém o valor teórico, o que é coerente com a verificação do simulador. A dispersão entre corridas é grande (coeficiente de variação de 12,8%) e a distribuição é assimétrica à direita (assimetria de 1,34), com algumas corridas de espera média alta.

Tabela 4*Estatísticas descritivas da espera média por corrida nas corridas piloto*

Estatística	Valor
Utilização do servidor (ρ)	0,90
Réplicas piloto (n)	30
Média (min)	9,112
Desvio-padrão (min)	1,166
Coeficiente de variação (%)	12,8
Mínimo (min)	7,258
1.º quartil (min)	8,296
Mediana (min)	8,837
3.º quartil (min)	9,508
Máximo (min)	12,591
Assimetria	1,34
Curtose (excesso)	2,21
Shapiro–Wilk W (p)	0,891 (0,005)
IC 95% da média (min)	[8,677; 9,548]
Valor teórico de W_q (min)	9,000
Erro relativo da média (%)	1,2

Nota. Resumo de 30 corridas piloto independentes da célula 3 ($N = 50.000$, $W = 1.000$). IC: intervalo de confiança de 95% da média (distribuição t). O valor teórico é o da fórmula de Pollaczek–Khinchine para M/M/1.

A Tabela 5 descreve os tempos de espera individuais de uma corrida piloto. A fração de clientes que não espera ($P(W_q = 0)$) e os percentis concordam com os valores teóricos da fila M/M/1, em que $P(W_q > t) = \rho e^{-(\mu-\lambda)t}$. A cauda é longa: em 1% dos casos a espera passa de 44,2 min, e a maior espera observada foi de 64,0 min.

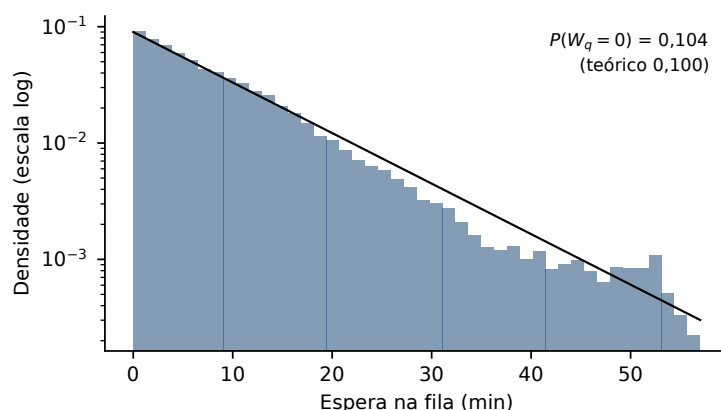
Tabela 5*Estatísticas descritivas dos tempos de espera individuais (primeira corrida piloto)*

Estatística	Empírico	Teórico (M/M/1)
Clientes analisados	49.000	—
$P(W_q = 0)$	0,104	0,100
Média (min)	8,13	9,00
Desvio-padrão (min)	8,96	9,95
Percentil 90 (min)	19,21	21,97
Percentil 99 (min)	44,22	45,00
Máximo (min)	64,0	—
Assimetria	2,02	—
Autocorrelação (defasagem 1)	0,99	—

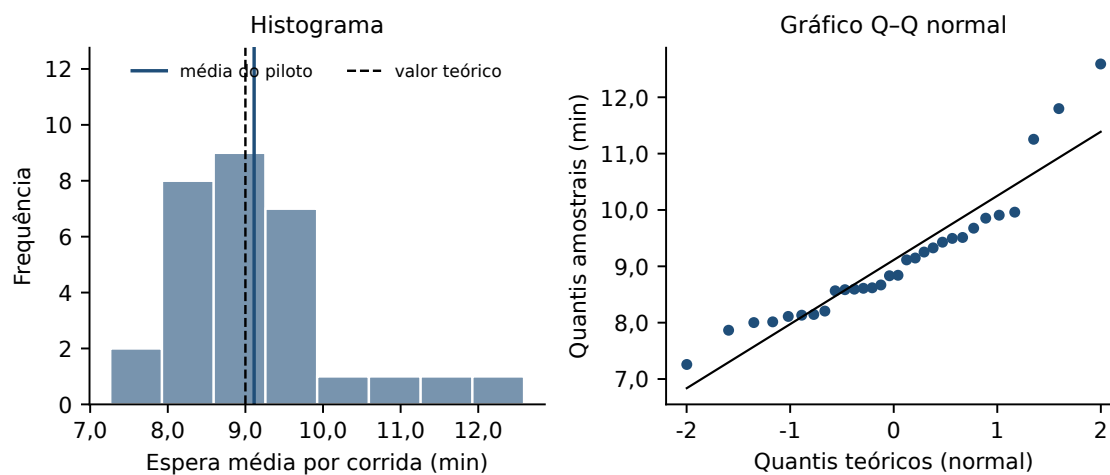
Nota. Clientes após o aquecimento. Valores teóricos da fila M/M/1. $P(W_q = 0)$ é a fração de clientes que encontra o servidor livre. — indica que não há valor teórico calculado.

Distribuições

A Figura 2 mostra que a distribuição das esperas individuais tem uma massa de probabilidade em zero e uma cauda exponencial, que segue a curva teórica. As médias por corrida, base da análise fatorial, aparecem na Figura 3. O teste de Shapiro–Wilk (Shapiro & Wilk, 1965) rejeitou a normalidade ($W = 0,891$, $p = 0,005$), e o gráfico Q–Q mostra afastamento na cauda superior, causado por poucas corridas de espera média alta. Na escala logarítmica, a assimetria cai para 0,96 e o teste não rejeita a normalidade ao nível de 5% ($p = 0,058$), embora com $n = 30$ o poder do teste seja limitado. Isso não decide a análise: apenas indica que a normalidade dos resíduos deve ser verificada no experimento, com a transformação logarítmica como candidata (seção “Projeto Fatorial 2^3r ”).

Figura 2*Distribuição das esperas individuais*

Nota. Histograma das esperas positivas de uma corrida piloto, com escala logarítmica no eixo vertical. A área total é ρ , pois a fração restante $(1 - \rho)$ tem espera zero. A linha preta é a densidade teórica M/M/1.

Figura 3*Distribuição das médias por corrida nas corridas piloto*

Nota. Histograma (à esquerda) e gráfico Q-Q normal (à direita) das 30 médias por corrida. A linha tracejada é o valor teórico e a linha cheia é a média do piloto.

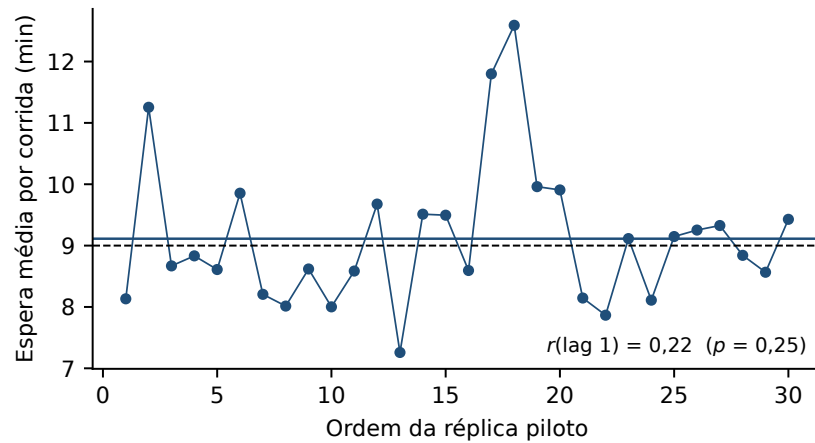
Visualizações

Independência entre réplicas. O projeto exige réplicas independentes. A Figura 4 mostra as médias por corrida na ordem das réplicas: não há tendência nem agrupamento evidente, e a correlação entre réplicas consecutivas foi $r = 0,22$

($p = 0,25$), sem indício de dependência (com $n = 30$, o poder é limitado). Isso é o esperado, porque cada réplica é uma execução completa com semente própria.

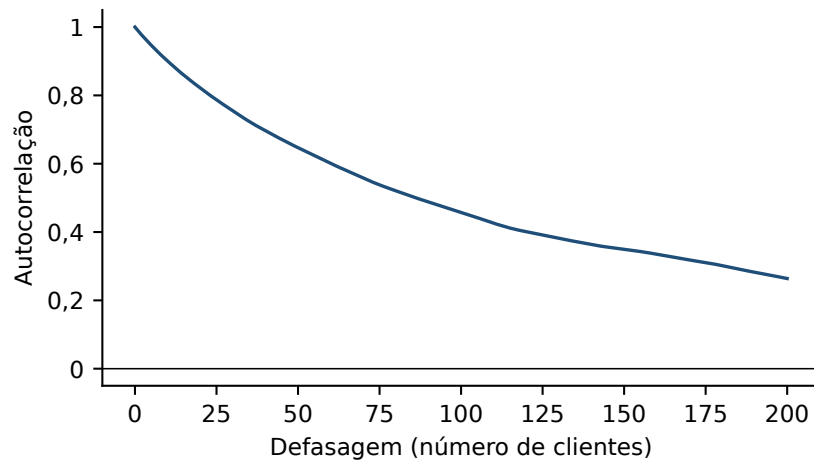
Figura 4

Média por corrida segundo a ordem das réplicas piloto



Nota. A linha cheia é a média do piloto e a tracejada é o valor teórico. r é a correlação de Pearson entre réplicas consecutivas.

Dependência entre clientes. As esperas de clientes consecutivos são fortemente correlacionadas: a autocorrelação de defasagem 1 foi 0,99 e a de defasagem 10 ainda foi 0,90 (Figura 5). Portanto, as esperas individuais não são observações independentes. Por isso a unidade de análise é a média por corrida, e as réplicas do projeto são execuções completas, e não clientes consecutivos nem partes de uma mesma corrida.

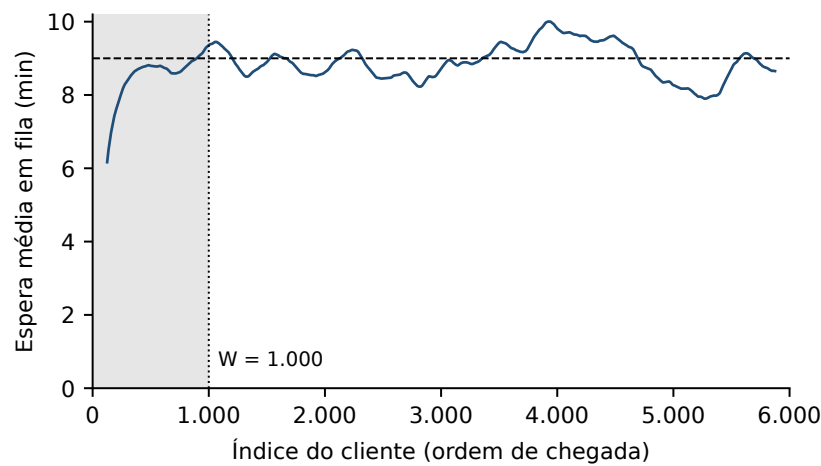
Figura 5*Autocorrelação das esperas individuais*

Nota. Função de autocorrelação amostral das esperas de uma corrida piloto, até a defasagem 200.

Período de aquecimento. Como o sistema começa vazio, os primeiros clientes esperam menos que os do regime estacionário. A Figura 6 mostra o procedimento gráfico de Welch (1983): a média entre 300 réplicas para cada índice de cliente, suavizada por média móvel de 250 clientes (ver também Kelton & Law, 1985; Law & Kelton, 2000). A média dos 500 primeiros clientes (7,42 min) ficou $-16,6\%$ em relação ao platô (8,90 min), e a curva se estabiliza antes de 1.000 clientes. Com $W = 1.000$, a diferença entre os clientes 1.001 a 2.000 e o platô foi de $-0,12 \pm 0,36$ min (erro-padrão), indistinguível de zero.

Figura 6

Procedimento gráfico de Welch para o período de aquecimento



Nota. Média entre 300 réplicas de 6.000 clientes, suavizada por média móvel de 250 clientes.

A linha tracejada é o valor teórico do regime estacionário; a região cinza é o aquecimento descartado ($W = 1.000$).

Viabilidade e decisões de projeto

A Tabela 6 compara corridas de 10.000 e de 50.000 clientes. Com 10.000 clientes, o coeficiente de variação foi de 17,2%, o que, com $r = 10$, daria um intervalo de confiança de cerca de $\pm 12,3\%$ sobre a média de uma combinação. Com 50.000 clientes, o coeficiente de variação caiu para 12,8% ($\pm 9,2\%$), a um custo de 0,45 s por corrida.

Tabela 6

Efeito do comprimento da corrida sobre a dispersão da resposta

N (clientes)	s (min)	CV (%)	Meia-largura do IC ($r = 10$)	Tempo/corrida (s)
10.000	1,508	17,2	$\pm 12,3\%$	0,088
50.000	1,166	12,8	$\pm 9,2\%$	0,445

Nota. Piloto da célula 3 ($W = 1.000$ em ambos os casos), com 30 corridas por linha. s : desvio-padrão das médias por corrida. CV: coeficiente de variação. Meia-largura do IC: $t_{0,975;9} s / (\sqrt{10} \bar{y})$, em porcentagem da média, para uma combinação com $r = 10$ replicações.

As decisões que decorrem da AED são:

- (1) usar $N = 50.000$ clientes por corrida, para reduzir a dispersão da resposta (se a precisão ainda for insuficiente, N ou r podem crescer, pois cada corrida custa menos de 1 s);
- (2) descartar $W = 1.000$ clientes, valor com margem sobre o ponto em que a curva de Welch se estabiliza;
- (3) usar a média por corrida como unidade de análise, com réplicas que são execuções completas e independentes;
- (4) verificar no experimento a normalidade e a homogeneidade de variâncias dos resíduos, considerando a transformação logarítmica;
- (5) executar as 80 corridas em cerca de 36 s, tempo compatível com o hardware disponível.

Considerações Finais e Próximos Passos

O piloto mostrou que o simulador é coerente com a teoria, que o aquecimento e o comprimento das corridas estão bem definidos, que as réplicas são independentes e que a média por corrida é assimétrica à direita, o que exigirá verificar a normalidade e a homogeneidade de variâncias dos resíduos do experimento. Nada disso avalia a hipótese geral, que depende do fatorial completo. Os próximos passos, na Entrega 2, são: (a) executar as 80 corridas na ordem aleatória do plano; (b) estimar os efeitos principais e as interações, com intervalos de confiança e testes de significância; (c) verificar as suposições com os resíduos; e (d) fazer a avaliação preliminar da hipótese.

Este estudo tem limitações que devem ser mantidas em mente na interpretação: os resultados valem para chegadas de Poisson, fila infinita e tempo de serviço com média fixa; $r = 10$ é o mínimo exigido, e, como o custo de cada corrida é baixo, o número de replicações pode ser aumentado se os intervalos de confiança ficarem largos; e a combinação com servidor duplo e serviço constante não tem verificação teórica exata.

Referências

- Box, G. E. P., & Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B (Methodological)*, 26(2), 211–252.
<https://doi.org/10.1111/j.2517-6161.1964.tb00553.x>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Jain, R. (1991). *The art of computer systems performance analysis: Techniques for experimental design, measurement, simulation, and modeling*. John Wiley & Sons.
- Kelton, W. D., & Law, A. M. (1985). The transient behavior of the M/M/c queue, with implications for steady-state simulation. *Operations Research*, 33, 378–396.
- Law, A. M., & Kelton, W. D. (2000). *Simulation modeling and analysis* (3^a ed.). McGraw-Hill.
- Lünsdorf, O., & Scherfke, S. (s.d.). *SimPy: Discrete event simulation for Python* (Versão 4.1.2). <https://simpy.readthedocs.io>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.
<https://doi.org/10.25080/Majora-92bf1922-00a>
- Montgomery, D. C. (2017). *Design and analysis of experiments* (9^a ed.). John Wiley & Sons.
- O’Neill, M. E. (2014). *PCG: A family of simple fast space-efficient statistically good algorithms for random number generation* (Technical report N. HMC-CS-2014-0905). Harvey Mudd College.
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10), e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>

- Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. *Proceedings of the 9th Python in Science Conference*, 92–96.
<https://doi.org/10.25080/Majora-92bf1922-011>
- Shapiro, S. S., & Wilk, M. B. (1965). An analysis of variance test for normality (complete samples). *Biometrika*, 52(3–4), 591–611.
<https://doi.org/10.1093/biomet/52.3-4.591>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272.
<https://doi.org/10.1038/s41592-019-0686-2>
- Welch, P. D. (1983). The statistical analysis of simulation results. Em S. S. Lavenberg (Ed.), *The computer performance modeling handbook* (pp. 268–328). Academic Press.
- Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>

Apêndice A

Plano de Aleatorização

O plano completo das 80 corridas está em `dados/plano_experimental.csv`. Ele é gerado por `codigo/plano_experimental.py` com a semente-mestre das corridas (`SEED_EXPERIMENTO = 20260922`) e a semente da permutação da ordem (`SEED_ORDEM = 20260923`). A Tabela A1 mostra as primeiras 12 corridas da ordem sorteada.

Tabela A1

Primeiras corridas da ordem de execução aleatorizada

Ordem	Corrida	Célula	Réplica	c	λ / Serviço	Chave da semente
1	24	3	4	1	0,9 / exponencial	23
2	62	7	2	1	0,9 / constante	61
3	56	6	6	2	0,5 / constante	55
4	26	3	6	1	0,9 / exponencial	25
5	73	8	3	2	0,9 / constante	72
6	55	6	5	2	0,5 / constante	54
7	53	6	3	2	0,5 / constante	52
8	15	2	5	2	0,5 / exponencial	14
9	78	8	8	2	0,9 / constante	77
10	9	1	9	1	0,5 / exponencial	8
11	59	6	9	2	0,5 / constante	58
12	71	8	1	2	0,9 / constante	70

Nota. A chave da semente identifica o filho de `SeedSequence.spawn` usado na corrida. O plano completo tem 80 linhas.

Apêndice B

Organização dos Arquivos e Reprodução

entrega1/
 codigo/ simulador.py, config.py, piloto_eda.py,
 plano_experimental.py, verificacao_simulador.py,
 executar_experimento.py, requirements.txt
 dados/ piloto_*.csv, plano_experimental.csv, verificacao_simulador.csv
 figuras/ figuras em PDF geradas por piloto_eda.py
 tabelas/ tabelas e macros numéricos em LaTeX (gerados)
 relatorio/ relatorio.tex, referencias.bib
 apresentacao/ apresentacao.tex
 Makefile

Para reproduzir os resultados desta entrega: `pip install -r codigo/requirements.txt`, seguido de `make all`, que executa o piloto, a verificação do simulador, o plano experimental e compila o relatório e a apresentação. As 80 corridas do experimento só serão executadas depois da aprovação da proposta, com `python codigo/executar_experimento.py`.